UNIVERSITY of HOUSTON
DEPARTMENT OF COMPUTER SCIENCE

# Anomaly Detection

Rakesh Verma

# Anomaly Detection

Outliers are instances which are dissimilar from the rest of the dataset.

Fundamental Assumption: Outliers are fewer than normal instances

Statistical Approaches: Let Interquartile Range,

IQR = Q3 – Q1, Q3 – 3rd quartile and Q1 – 1$^{st}$ quartile,

outliers are points beyond Q3 + 1.5IQR and below Q1 – 1.5IQR

Methods based on mean and standard deviation also exist, but the mean itself is sensitive to outliers, hence not preferred.

## Proximity-based approaches

Outliers are significantly dissimilar from the dataset than other non-outliers.

Distance-based

If there are not enough points close to an instance, then it is an outlier.

Density-based

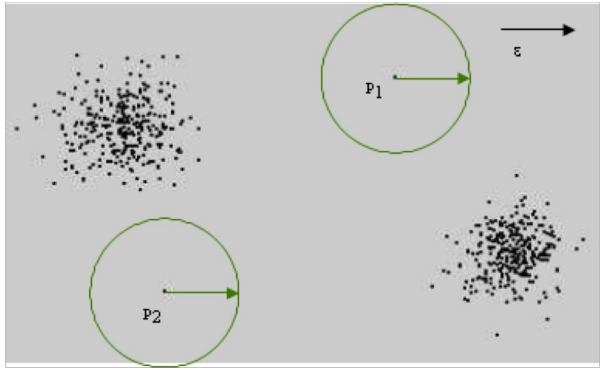
If the density of an instance is much lower than its nearby instances, it is an outlier.

# DB(ε,π) Outliers

Given a radius ε and percentage π

A point is an outlier if at most π percent of all other points are closer than ε.

$$OutlierSet(\varepsilon,\pi) = \{p \mid \frac{Card(\{q \in DB \mid dist(p,q) < \varepsilon\})}{Card(DB)} \leq \pi\}$$



# Distance-based Approaches

## Index-based

- Compute distance range join using spatial index structure
- Exclude point from further consideration if its ε-neighborhood contains $n\pi$ points.

## Nested loop-based

- Divide buffer into two parts
- Compare all points in first part with all points in second part

## Grid-based

- Build a grid such that points in the same cell are at most away from each other
- Only compare points in neighboring cells.

# Deriving Intensional Knowledge

Find the minimal set of attributes responsible for meeting outlier criterion.

| Player Name | Power-play Goals | Short-handed Goals | Game-winning Goals | Game-tying Goals | Games Played |
|---|---|---|---|---|---|
| MARIO LEMIEUX | 31 | 8 | 8 | 0 | 70 |
| JAROMIR JAGR | 20 | 1 | 12 | 1 | 82 |
| JOHN LECLAIR | 19 | 0 | 10 | 2 | 82 |
| ROD BRIND'AMOUR | 4 | 4 | 5 | 4 | 82 |

Derived Intensional Knowledge →

```
MARIO LEMIEUX:
  (i)  An outlier in the 1-D space of Power-play goals
  (ii) An outlier in the 2-D space of Short-handed goals and
       Game-winning goals
       (No player is exceptional on Short-handed goals alone;
        No player is exceptional on Game-winning goals alone.)
ROD BRIND'AMOUR:
  (i)  An outlier in the 1-D space of Game-tying goals
JAROMIR JAGR:
  (i)  An outlier in the 2-D space of Short-handed goals and
       Game-winning goals
       (No player is exceptional on Short-handed goals alone;
        No player is exceptional on Game-winning goals alone.)
  (ii) An outlier in the 2-D space of Power-play goals and
       Game-winning goals
```

# kNN-based Approaches

- k-Nearest Neighbors (kNN) selects k nearest points using a distance measure.
  - Using these distances from its *k* nearest points, we can calculate the outlier score.
  - Alternatively, aggregate all kNN outlier scores from 1 to *k* as the outlier score.
- Loop-Based
  - For each instance, calculate its distance to every other point.
  - Sort and pick the closest *k* points.
- Partition-Based
  - Cluster data first
  - Perform kNN within each cluster
    - This allows us to skip calculation of distances between far clusters.

# Distance-based Top-*n* Outliers

## Linearization

- Map *n*-dimensional space to a 1-dimensional space using space filling curve.
- Partition space-filling curve into micro clusters
- Use kNN with those micro clusters to identify outliers.
  - The basic space-filling curves is a curve that bijectively map from 1D space to 2D.
  - Generalizing the idea allows this method to use any-dimensional space filling curve on any dataset.

## ORCA

- Randomly pick *n* outliers and scan forward.
  - If you find a point with higher outlier score than a current outlier, prune the lowest score
- Works with any scoring measure

## RBRP

- Use pruning method with micro-cluster kNN method
- Prunes are more impactful and happen less often

# Distance-based Top-*n* Outliers (Continued)

## In-degree, Graphical Method

- Construct a graph for kNN
  - Vertices are instances
  - Directed edges from an instance *p* to its k-Nearest Neighbors
- Vertices with in-degree less than a threshold *T* are outliers.
  - i.e. points with few neighbors are outliers

## Resolution-based Outlier Factor

$$ROF(p) = \sum_{Rmin \le r \le Rmax} \frac{clusterSize_{r-1}(p) - 1}{clusterSize_r(p)}$$

- Points are either outliers or in clusters.
- Select Rmin and Rmax as extrema for cluster count.
- For a given point *p*,
  - Sum over the percentage change in *p*'s cluster as we allow more clusters from Rmin to Rmax.

# Density-based Approaches

Use the density at a current point as the points outlier score.

Assume that density around normal data objects are similar to their neighbors.

Assume that density around outlier data objects are dissimilar to their neighbors.

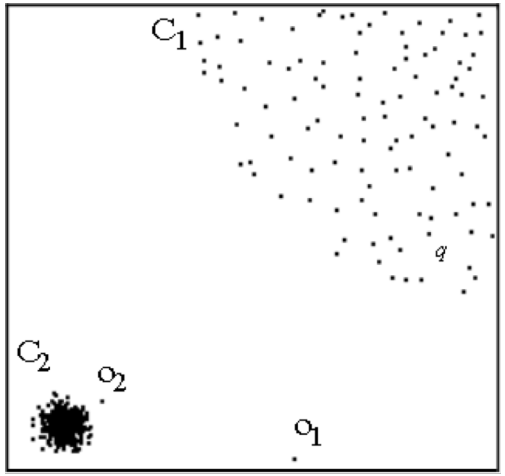
# Density-based Approaches

## Local Outlier Factor

Use relative densities of nearby points to determine outliers.

DB(ε,π) method fails to identify $o_2$ as an outlier without classifying all of $C_1$ as outliers.

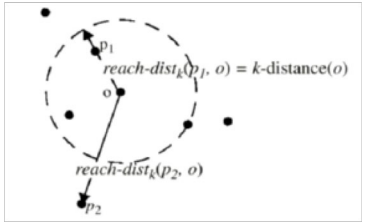
kNN approaches have difficulty with handling two different distances as well.




$$reach-dist_k(p,o) = \max\{k-\text{distance}(o), dist(p,o)\}$$

$$lrd_k(p) = 1/\left(\frac{\sum_{o \in kNN(p)} reach-dist_k(p,o)}{Card(kNN(p))}\right)$$

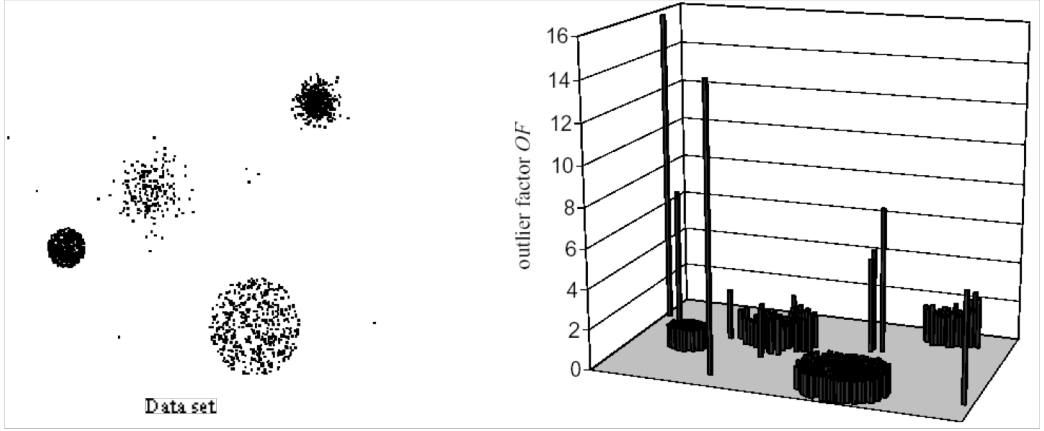
$$LOF_k(p) = \frac{\sum_{o \in kNN(p)} \frac{lrd_k(o)}{lrd_k(p)}}{Card(kNN(p))}$$



# Properties of Local Outlier Factor

Local Outlier Factor is approximately 1 in a cluster.

Local Outlier Factor is far greater than 1 for outliers.



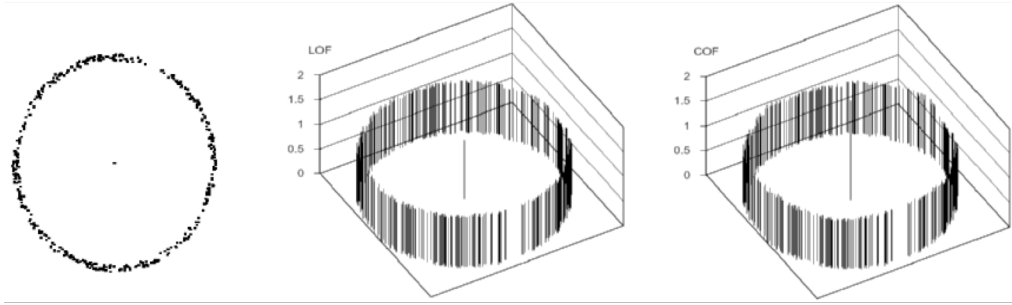
# Variants of Local Outlier Factor

## Mining Top-*n* local outliers

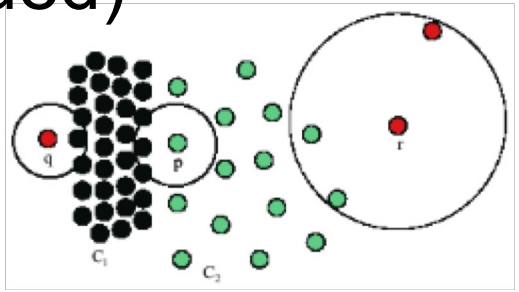
- Use BIRCH to construct clusters
- Derive bounds for reachability-distances, lrd-values, and LOF values inside clusters
- Sort dataset by LOF values
- Prune clusters which cannot contain outliers by the constraints places on LOF, lrd, and reachability bounds.
- Repeat the process on the pruned set

## Connectivity-based outlier factor

- Treat low-density and no-density differently.
- Works where isolated points would otherwise be difficult to discern from low density non-outliers.



# Variants of Local Outlier Factor (Continued)




## Influenced Outlierness (INFLO)

Close proximity clusters make it difficult for LOF to provide good results

INFLO uses the notion of influence space to better describe neighborhoods around points

The influence space around a point includes its k-Nearest Neighbors and all points for which it is one of its k-Nearest Neighbors.

$RNN_k$ is the reverse nearest neighbors.

$$\mathrm{IS}_k(p) = RNN_k(p) \cup NN_k(p)$$

In the figure to the right, you see *p*'s three nearest neighbors are black, but it is the nearest neighbor of four green points.

$$\mathrm{INFLO}_k(p) = \frac{den_{avg}(IS_k(p))}{den(p)}$$

# Properties of Influenced Outlierness (INFLO)

Similar to LOF

Influenced Outlierness is close to 1 when in a cluster

Influenced Outlierness is much larger than 1 when an outlier.

# Local Outlier Correlation Integral (LOCI)

Use the ε-neighborhood model instead of the kNN

An ε-neighborhood is all the points within ε of the point in question

Local density a point is the number of points in its ε-neighborhood.

Given an α, average neighborhood density of a point is calculated as the sum of the local (αε)-densities of all its neighbors.

$$den(p,\varepsilon,\alpha) = \frac{\sum_{q \in N(p,\varepsilon)} Card(N(q,\alpha \cdot \varepsilon))}{Card(N(p,\varepsilon))}$$

The Multi-granularity Deviation Factor (MDEF) is defined as follows.

$$MDEF(p,\varepsilon,\alpha) = \frac{den(p,\varepsilon,\alpha) - Card(N(p,\alpha \cdot \varepsilon))}{den(p,\varepsilon,\alpha)} = 1 - \frac{Card(N(p,\alpha \cdot \varepsilon))}{den(p,\varepsilon,\alpha)}$$

# Multi-granularity Deviation Factor (MDEF)

Multi-granularity Deviation Factor is 0 for points in a cluster.

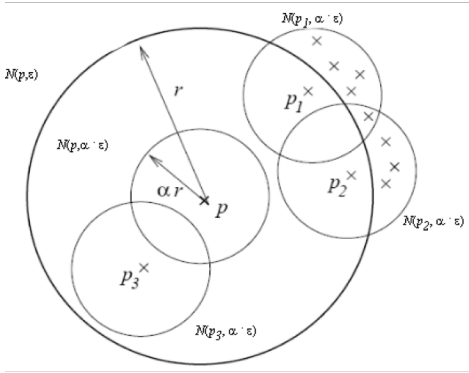
Multi-granularity Deviation Factor is greater than 0 for outliers.

Alternatively

σMDEF(p,ε,α) is the normalized standard deviation of the densities of all points from N(p,ε)

Points further than 3 standard deviations away are outliers.

MDEF > 3σMDEF(p,ε,α) → Outlier

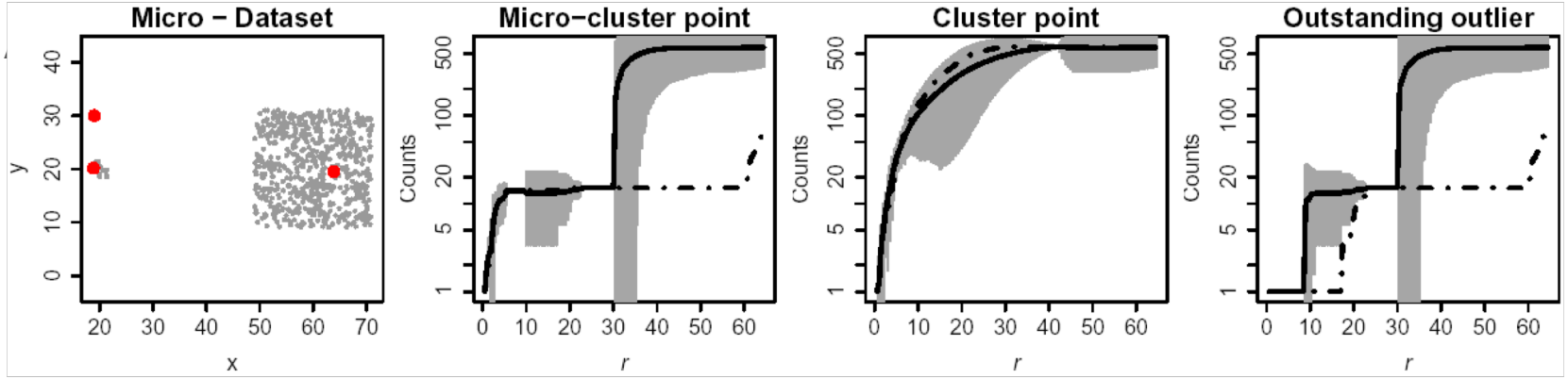


# Local Outlier Correlation Integral (LOCI)

All values of ε are tested, thus automatically determined

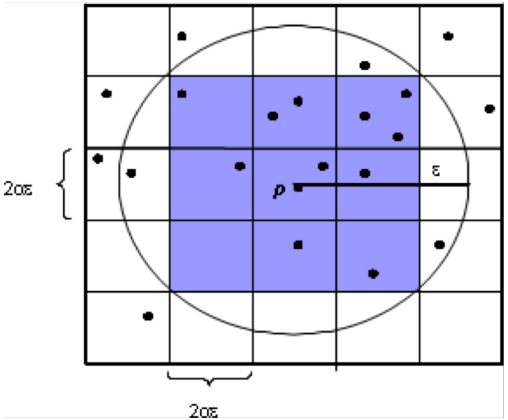
In fact, the entire method is automatic and data-driven.

Deals with both local density and multiple granularities



# aLOCI

- Approximate the ε-neighborhood used in LOCI with a grid.
  - Each cell is square-width 2αε.
  - Each cell sits inside the ε-neighborhood of its members.
- During iterative testing of different values of ε
  - Use a quadtree for to describe values ε / 2
    - This optimization efficiently separated every grid cell into four grid cells without much other modification.
- In the equation to the right, $c_j$ is the number of instances of the grid cell *c*.
- ζ(p, ε) is the set of grid cells inside the ε-neighborhood.




$$Card(N(q,\alpha\cdot\varepsilon)) = \frac{\sum_{c_j\in\zeta(p,\varepsilon)} c_j^{\,2}}{\sum_{c_j\in\zeta(p,\varepsilon)} c_j}$$

# Clustering-Based Outlier Detection

An object is an outlier if it doesn't belong to a cluster (or belongs to a rag-bag)

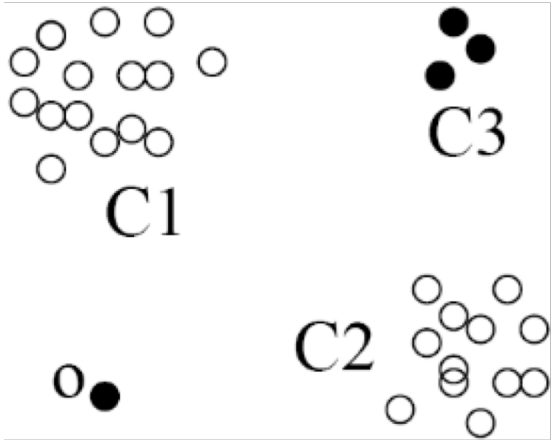
Cluster-based local outlier factor (CBLOF)

CBLOF(p) = (size of the of cluster of p)(similarity between p and closest large cluster)

Large cluster could be *p*'s cluster if it's large.

In the figure to the right, o is an outlier because it has low similarity with its nearest large clusters.

In the figure to the right, the points in C3 are outliers because their distance to the closest large cluster is vast and there's only 3 of them.



# Properties of Cluster-based local outlier factor

- Pros
  - Trains on unlabeled data
  - Works well on many types of data
  - Once clusters are computed, cluster centers and sizes are the only things involved in calculations.
- Cons
  - Heavily dependant on clustering method
    - Therefore, it shoulders the high computation cost of clustering
    - Fixed-width clustering can be used as an O(cn) method where c is the cluster count
      - May or may not produce sufficient quality clusters, but will produce them quickly

# One class learning based anomaly detection

Anomaly detection can also be based on one-class learning algorithms, e.g., one-class Support Vector Machines.

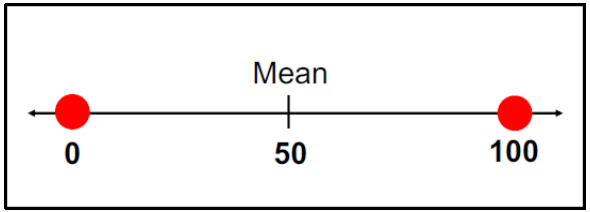
Usually better to design models for the normal class, since attacks keep changing.

Also data may be easier to obtain for the normal class.

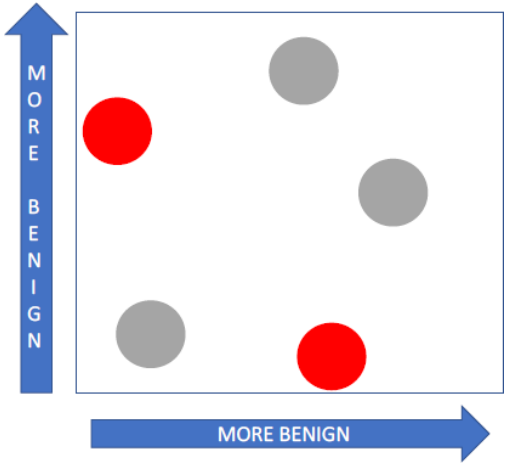
(Will revisit this under machine learning for security module)

# Limitation of Standard Techniques

- Require hyperparameter tuning
- **Direction-agnostic**(standard dev of +3 just as anomalous as -3)




- Alert if anomalous in only one dimension




Taken from https://www.usenix.org/sites/default/files/conference/protected-files/usenixsecurity17_slides_grant_ho.pdf

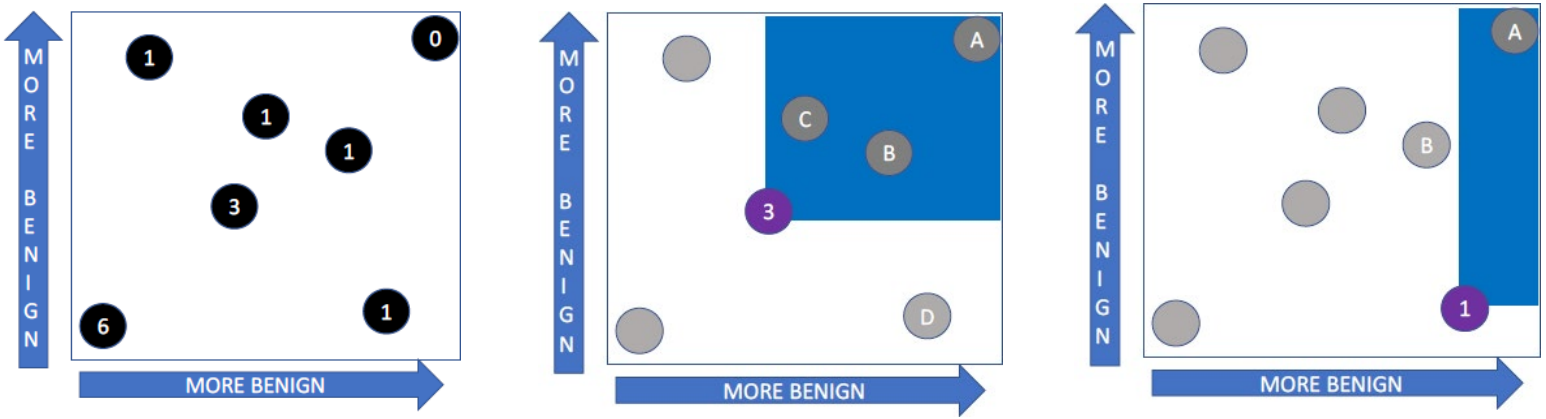
# Directed Anomaly Scoring (DAS)

- simple, new method that overcomes these 3 problems
- Steps:
  1. Security analysts w/ limited time: specify ***B*** = alert budget
  2. For set of events, assign each event a “suspiciousness” score
  3. Rank events by their “suspiciousness”
  4. Output the ***B*** most suspicious events for security team

Taken from https://www.usenix.org/sites/default/files/conference/protected-files/usenixsecurity17_slides_grant_ho.pdf

# Directed Anomaly Scoring

- Score(Event X) = # of other events that are as **benign** as X in ***every*** dimension
  - i.e., Large score = many other events are more benign than X




Taken from https://www.usenix.org/sites/default/files/conference/protected-files/usenixsecurity17_slides_grant_ho.pdf

# DAS: Application on SpearPhishing Email Detection

- Real-time detector on 370 million emails over ~4 years
- Ran detector w/ total budget of **10 alerts/day**
  - Practical for LBL's security team (~240 alerts/day typical)
- Detected **17 / 19** spearphishing attacks (89% TP)
  - 2 / 17 detected attacks were ***previously undiscovered***
- Best classical anomaly detection: **4/19** attacks for same budget
  - Need budget >= **91 alerts/day** to detect same # of attacks as DAS

Taken from https://www.usenix.org/sites/default/files/conference/protected-files/usenixsecurity17_slides_grant_ho.pdf

# References

1. Ho, Grant, et al. "Detecting Credential Spearphishing Attacks in Enterprise Settings." *USENIX security symposium*. 2017

2. R. Verma and D. Marchette, Cybersecurity Analytics, Chapman & Hall/CRC Press, 2019.